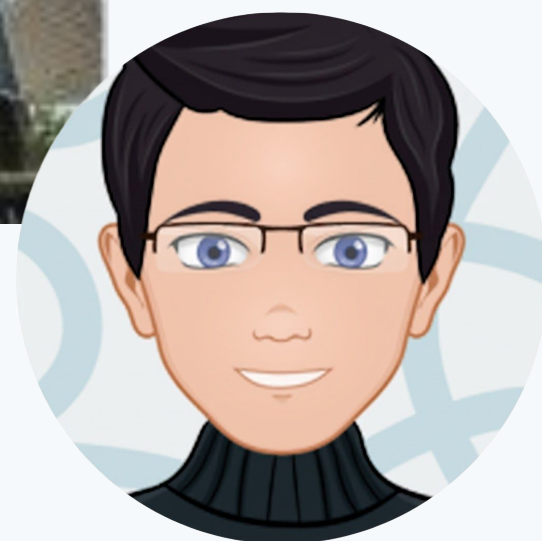


# KServe and the Next Step for LLM Workloads: LLMInferenceService in Context

Jooho Lee · Principal Software Engineer / Red Hat

---



## *Jooho Lee (Jay)*

Jooho Lee is a Toronto-based technology professional with over a decade at Red Hat, focusing on cloud-native platforms and AI. He began in middleware support and later consulted on OpenShift, helping customers design and operate Kubernetes platforms. Now on the OpenShift AI Model Serving team, he integrates KServe and LLM-D into Open Data Hub for production-grade, Kubernetes-native AI/ML deployments. He is also a KServe committer, contributing to model serving and LLM inference..

LinkedIn: [www.linkedin.com/in/jooho](https://www.linkedin.com/in/jooho)

Github: <https://github.com/jooho>

---

01

# Welcome LLM-D to the CNCF

New Incubating project LLM-D

---

# Welcome llm-d to the CNCF: Evolving Kubernetes into SOTA AI infrastructure

Posted on March 24, 2026 by Carlos Costa (IBM Research), Clayton Coleman (Google), and Rob Shaw (Red Hat)

We are thrilled to announce that **llm-d** has officially been accepted as a Cloud Native Computing Foundation (CNCF) Sandbox project!

---

# Do you remember vLLM??



[Docs](#) [Blog](#) [Events](#) [Contact](#)

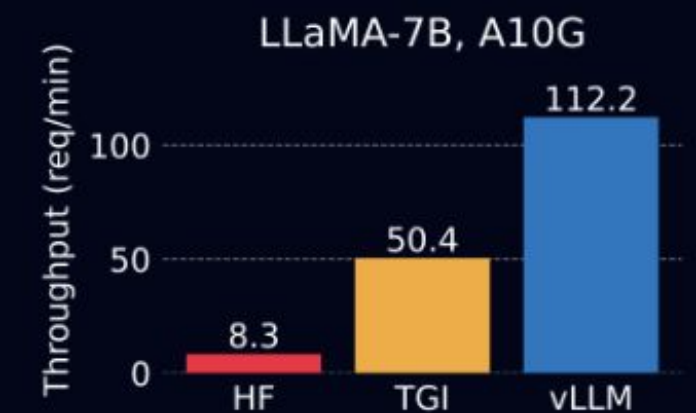
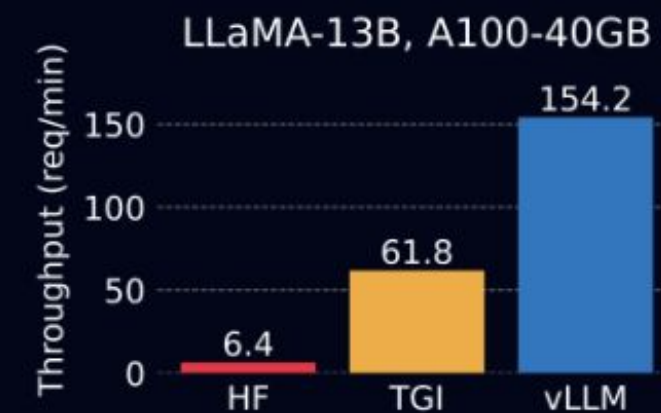
Blog

## vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention

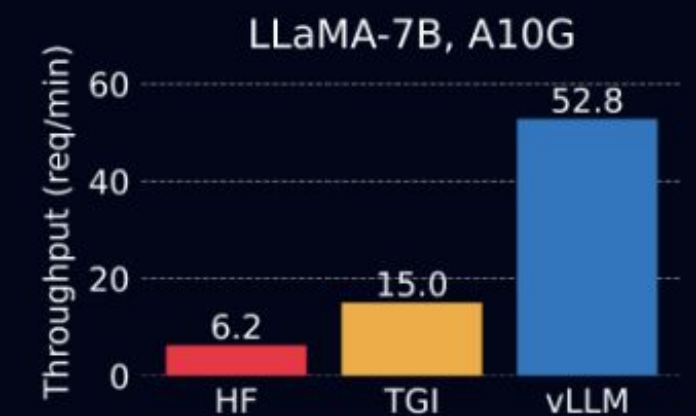
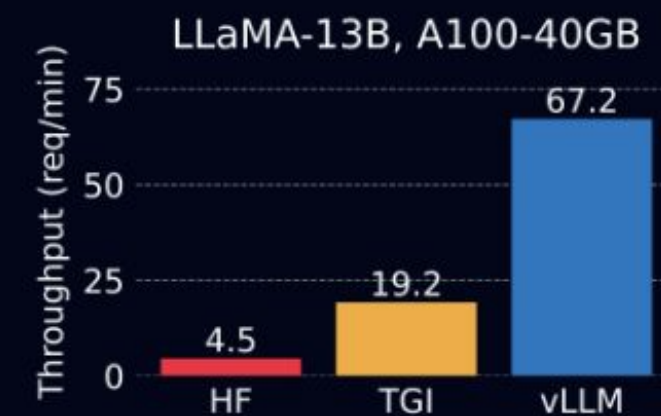
June 20, 2023 8 min read

Woosuk Kwon\*, Zhuohan Li\*, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Yu, Joey Gonzalez, Hao Zhang, and Ion Stoica (\* Equal Contribution)

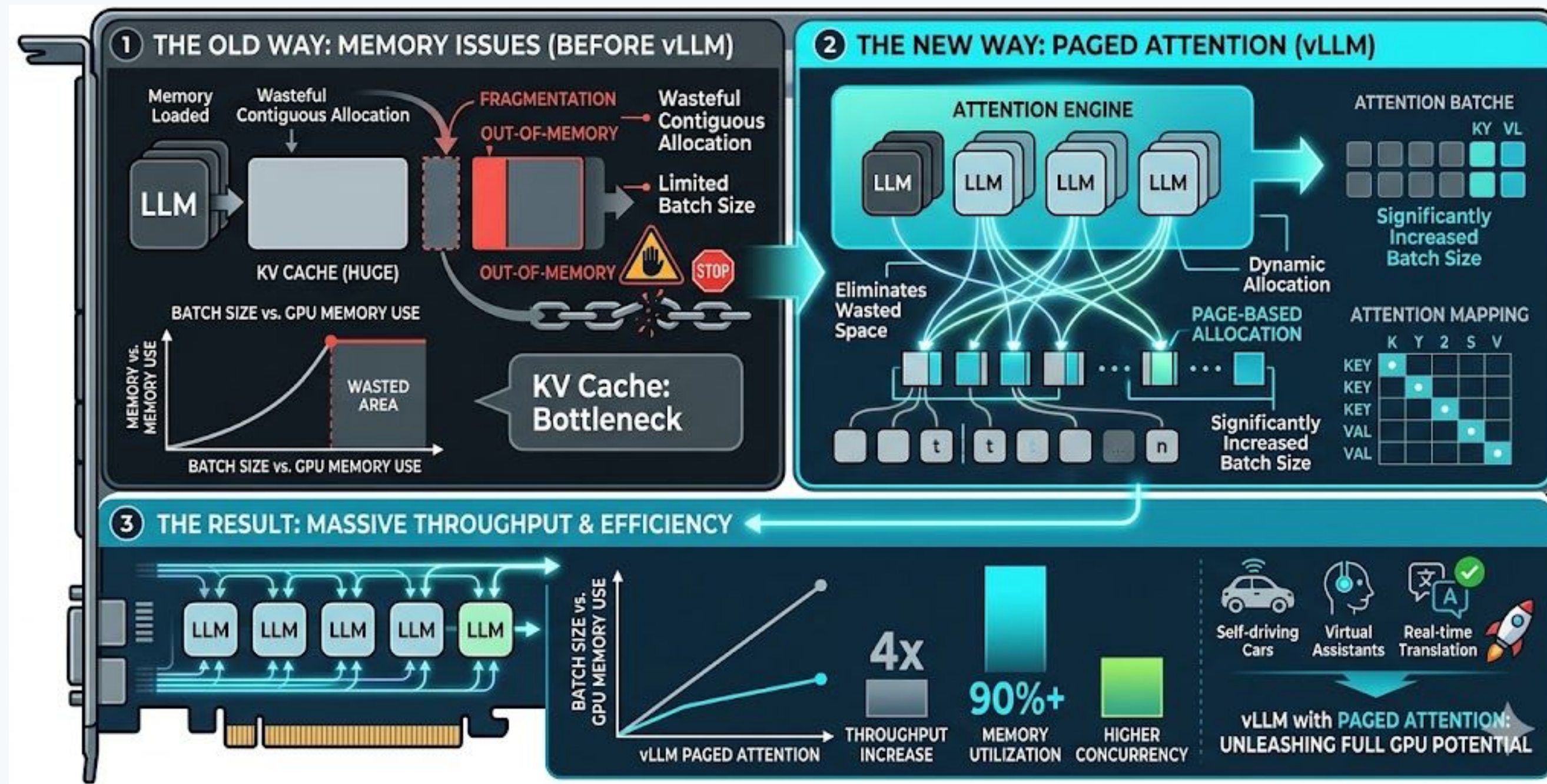
#performance



serving throughput when each request asks for *one output completion*. vLLM achieves 14x - 24x higher throughput than HF and 2.2x - 2.5x higher throughput than TGI.



# Improving KV Cache Efficiency with vLLM Paged Attention



# The Wall of Statefulness

## Blocking LLM Scaling



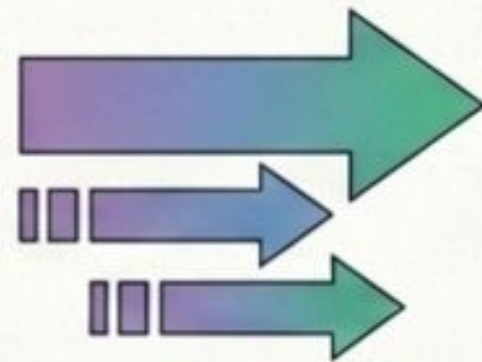
### 1. 'KV Cache' Bottleneck Phenomenon

Memory fragmentation occurs when scaling to multi-nodes. Latency spikes due to active session eviction (Preemption).



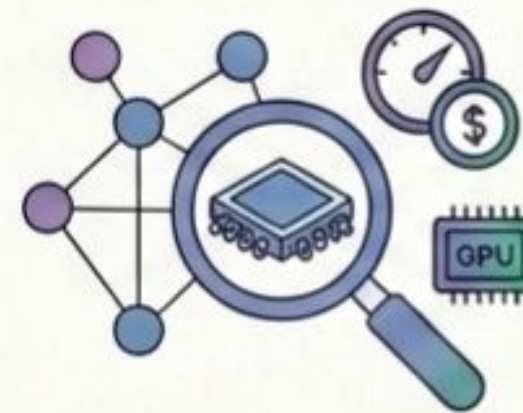
### 2. Inefficient Load Balancing

Existing L4/L7 load balancers are "LLM-blind", unaware of GPU memory pressure or queue length.



### 3. Lack of Request Prioritization

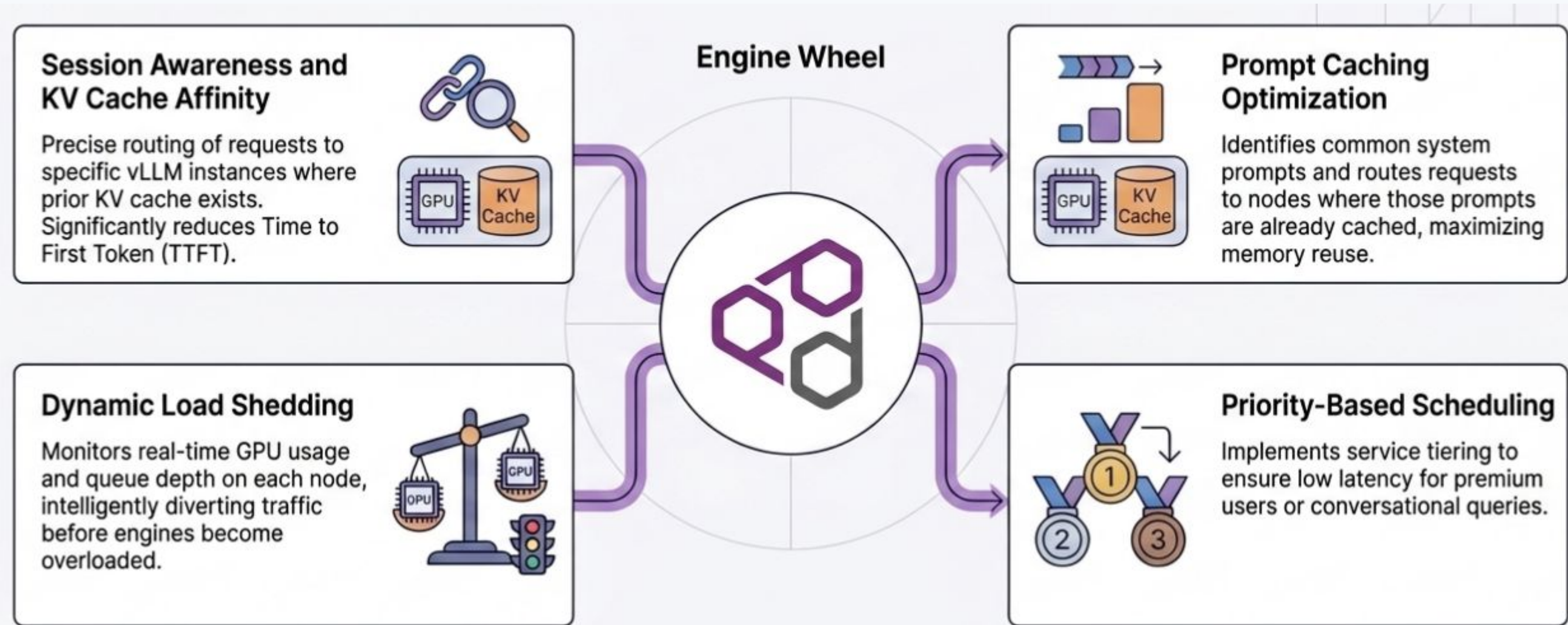
Due to FIFO processing, short interactive chatbot requests are stuck behind long batch jobs, collapsing user experience (UX).



### 4. Scalability and Visibility Gap

Single engine cannot handle global rate limiting, cost tracking, and failover between multi-GPU clusters.

# Innovation at the Orchestration Level: LLM Native Router (LLM-D)



# LLM-D

llm-d is a high-performance distributed inference serving stack optimized for production deployments on Kubernetes.

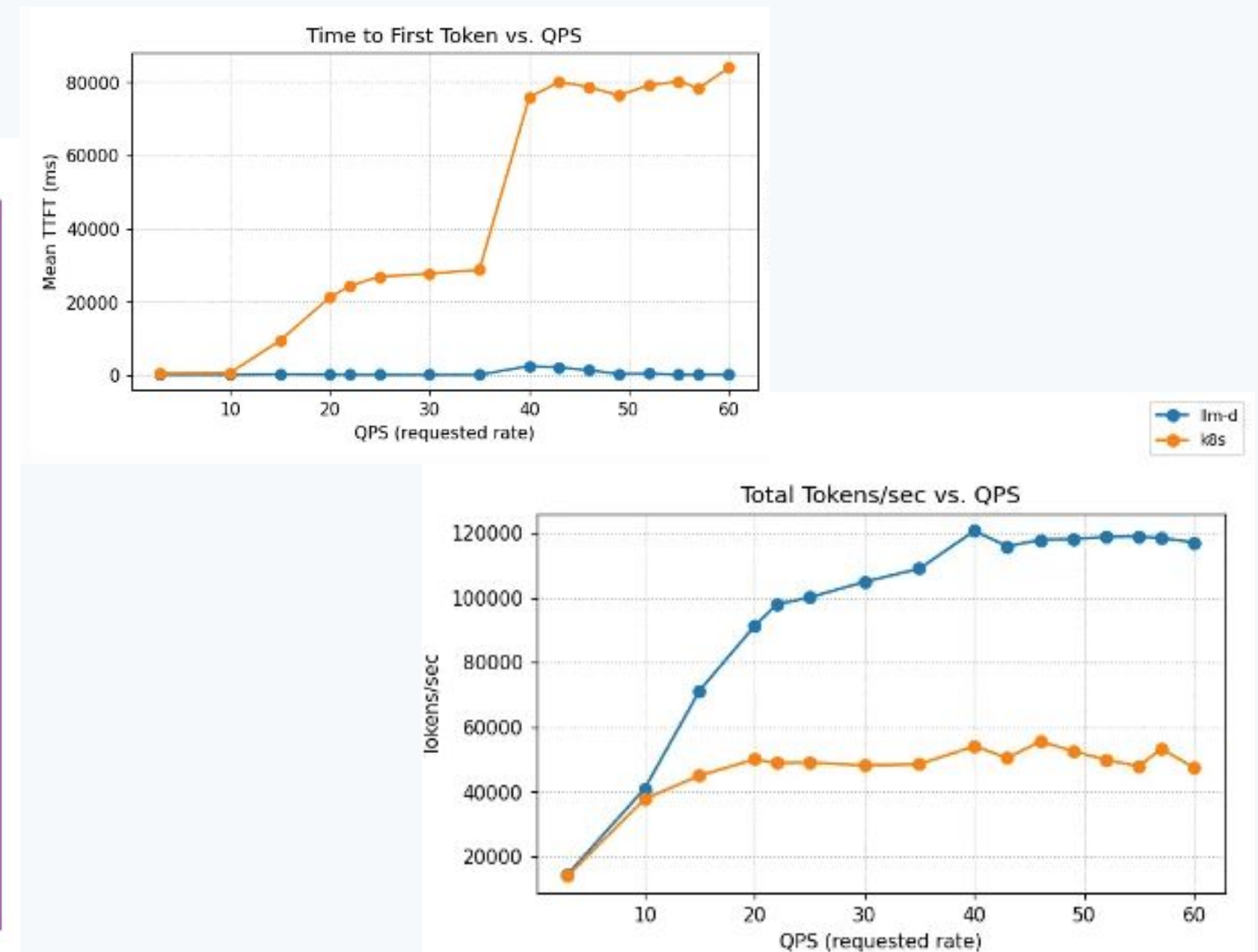
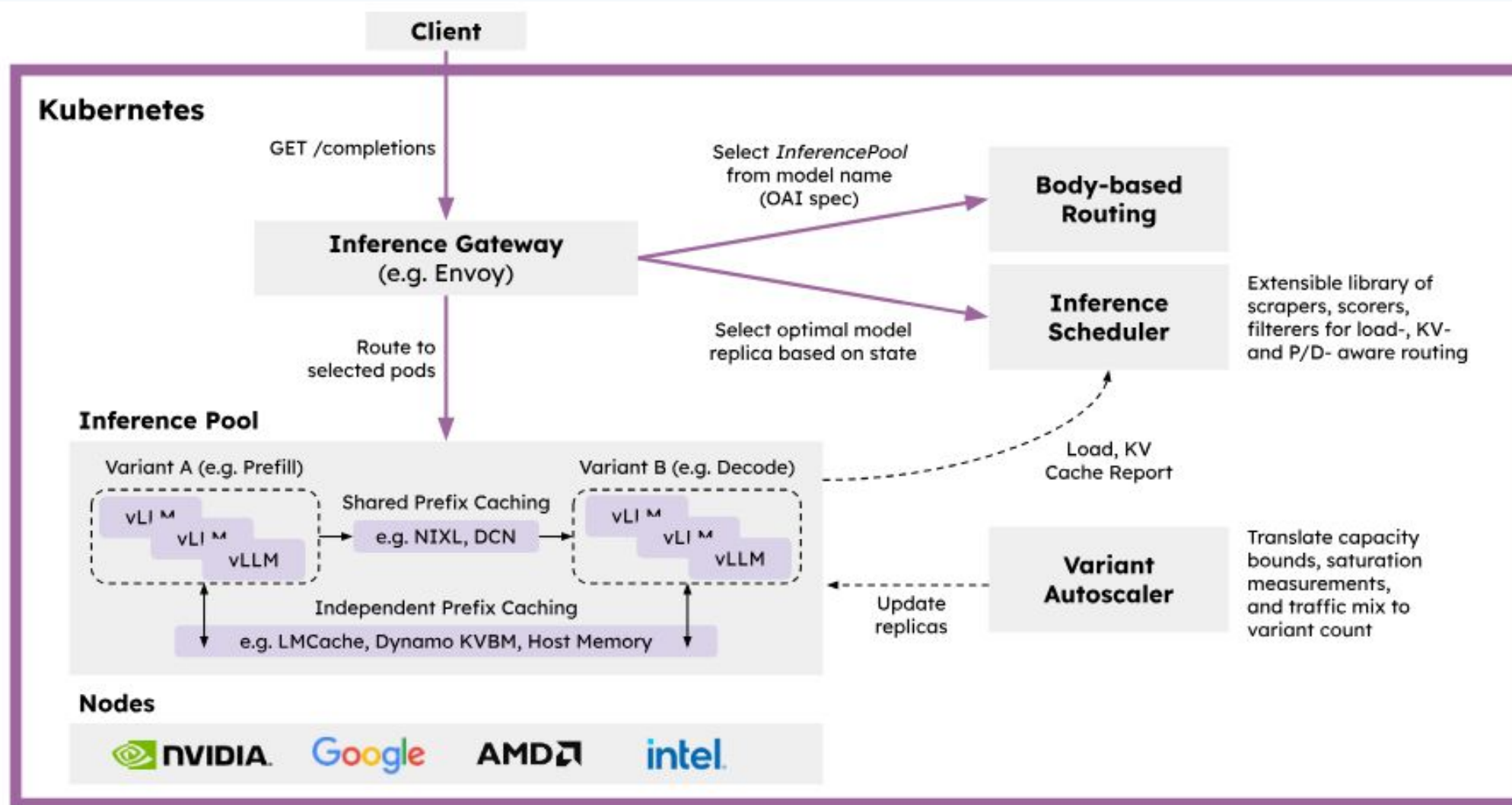


Figure 1: TTFT and throughput vs QPS on Qwen3-32B (8xvLLM pods, 16xNVIDIA H100).

02

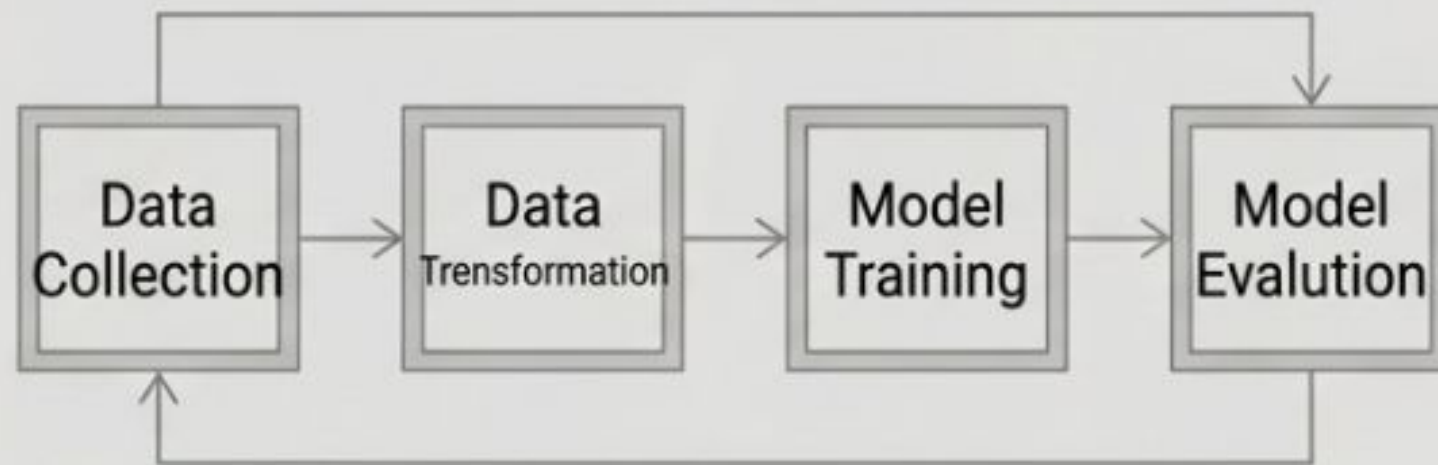
# KServe is evolving

We accepted LLMD with LLMISVC

---

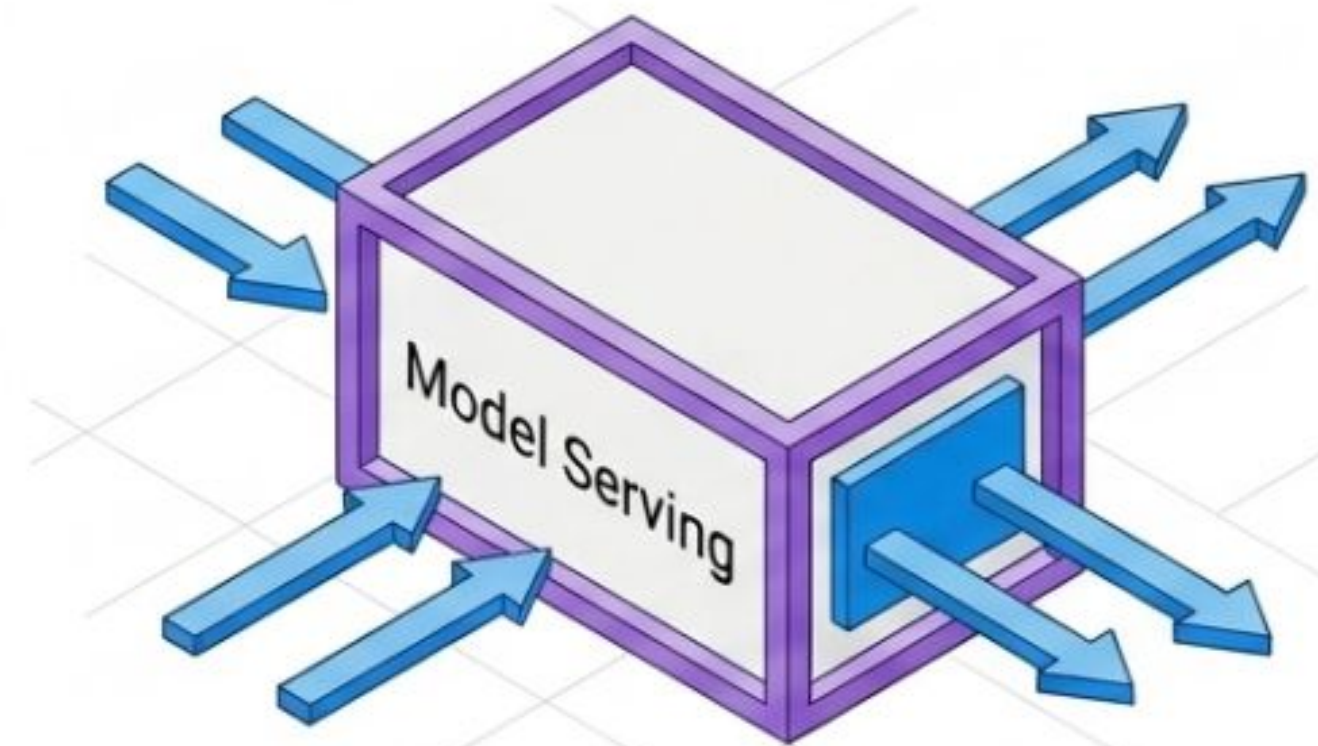
# The Model Lifecycle

## Model Creation - Training



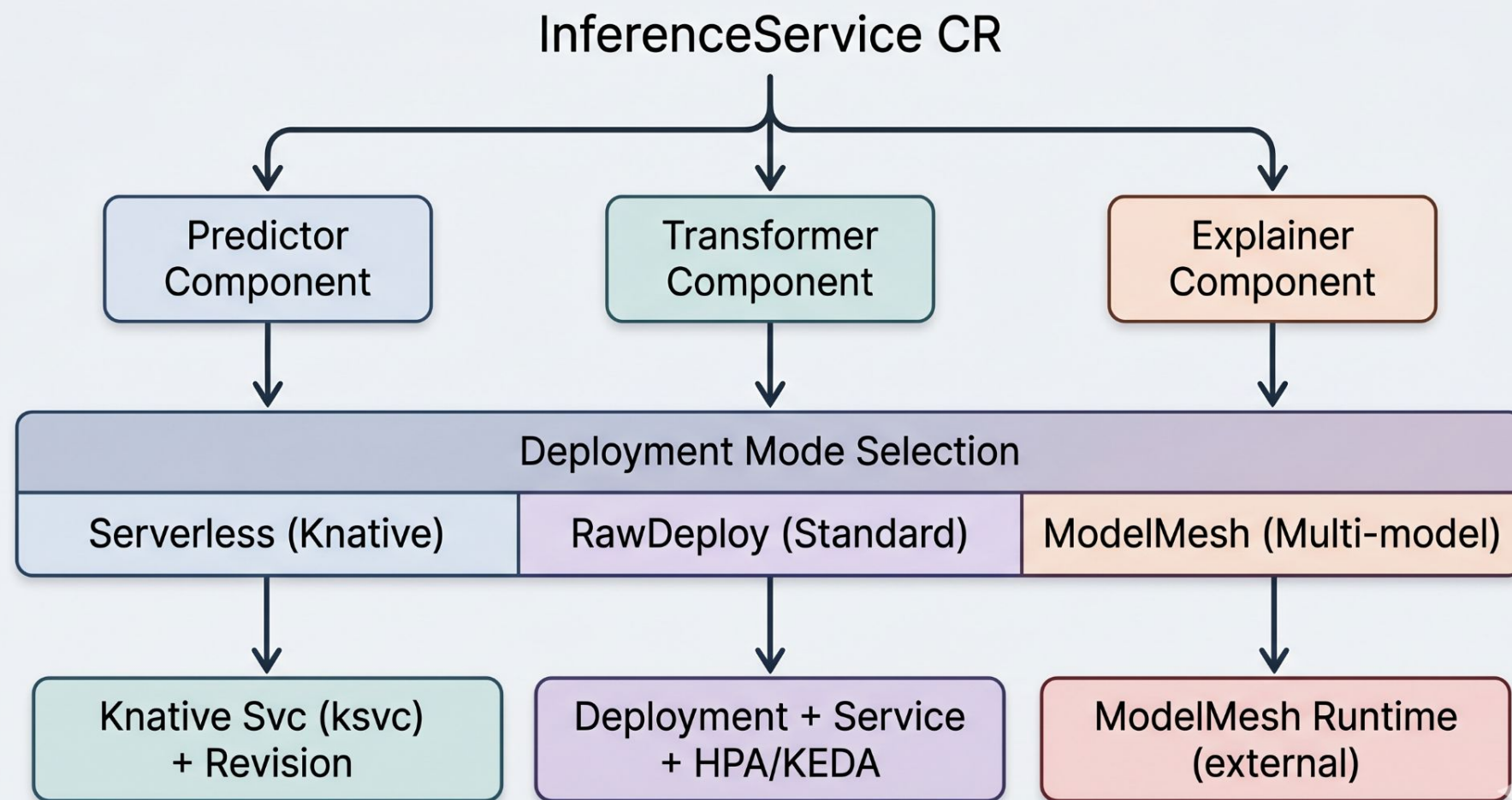
- Focus: Intensive resource investment and iterative operations.

## Model Consumt - Inference (KServe Domain)

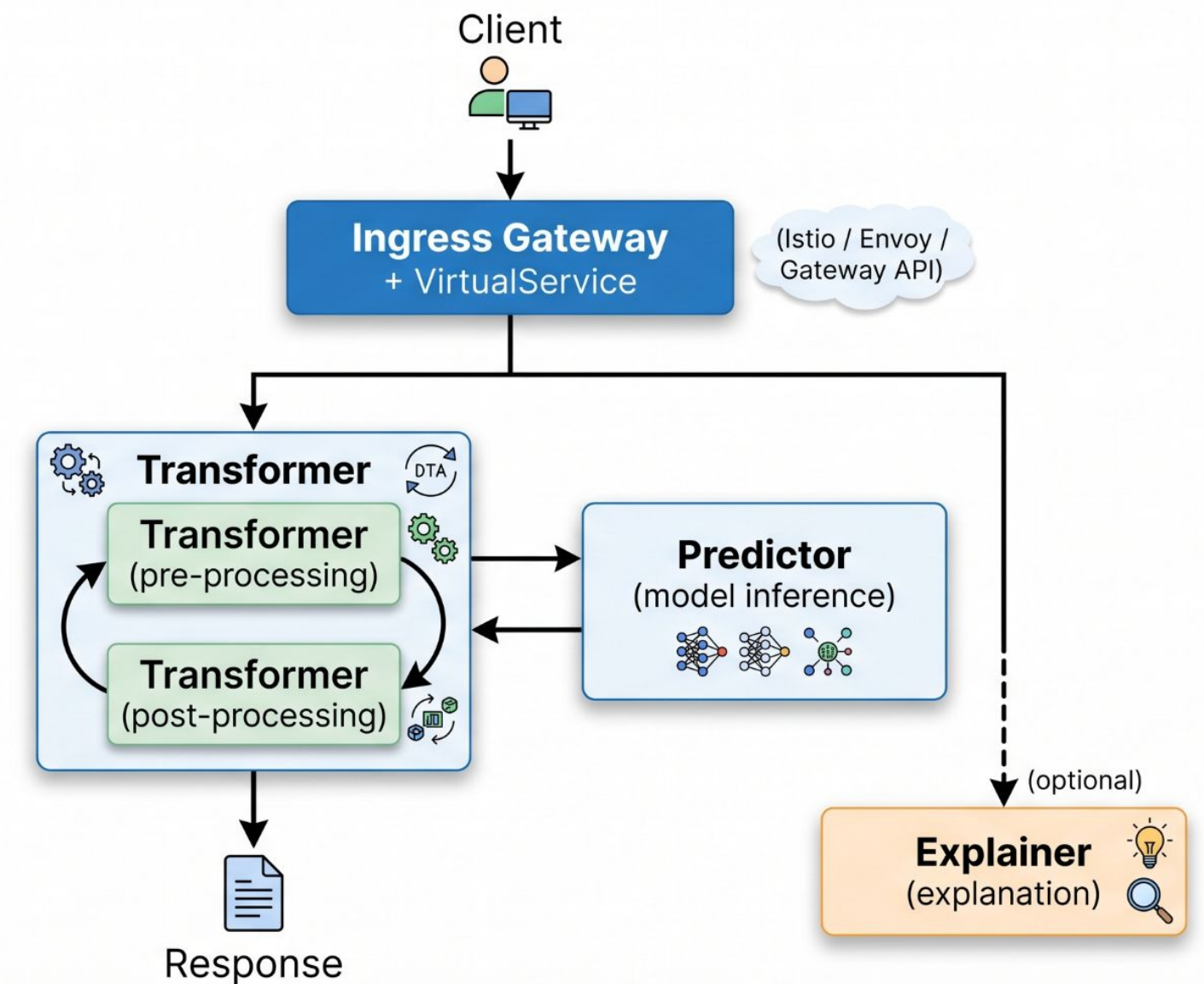


- Focus: Scalability, low latency, and real-time traffic processing.

# Predictive AI with KServe



## Request Flow



# Evolution of AI Serving Paradigm:

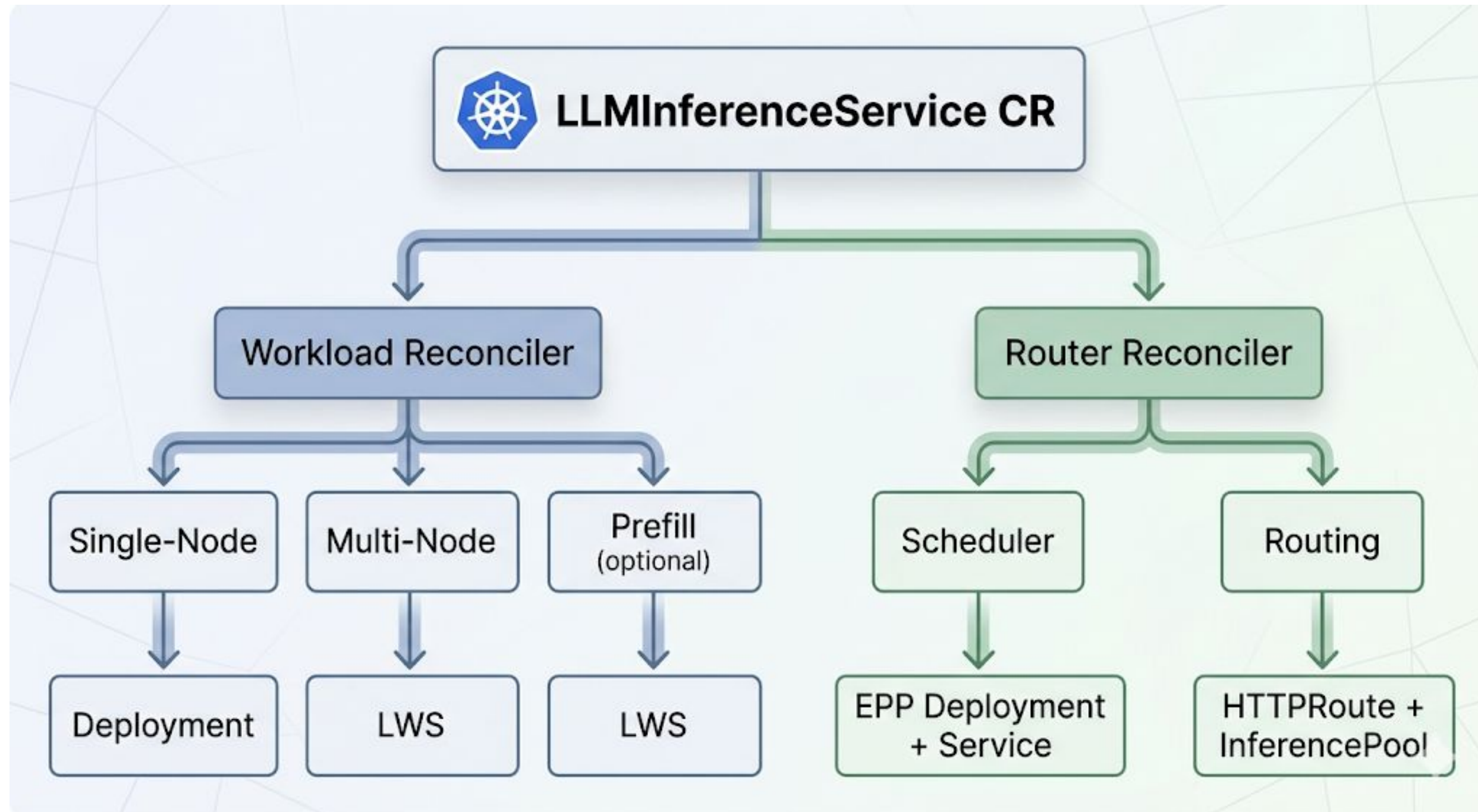
## Predictive vs Generative AI

|  Predictive AI |                                                                                |  Generative AI / LLM<br>(Generative AI) |  |
|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|--|
| Goal                                                                                            | Predict Future Outcomes & Trend Analysis                                       | Generate Creative Text, Code, Translations                                                                                 |  |
| Input/Output                                                                                    | Structured Data -> Probability & Classification Results                        | Large-scale Text/Prompt -> Streaming-Based Token Generation                                                                |  |
| Workload Characteristics                                                                        | Stateless, Short Computation Time, Optimized for Traffic-Based Load Balancing. | Highly Stateful, Huge KV Cache Memory Footprint, Standard Load Balancers Not Applicable.                                   |  |

# CRD Comparison

|  InferenceService<br>(Classic ML)                                                                                                                                |  LLMInferenceService<br>(GenAI)                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>apiVersion: serving.kserve.io/v1beta1 kind: InferenceService metadata:   name: sklearn-iris spec:   predictor:     model:       modelFormat:         name: sklearn       storageUri: gs://kfserving- examples/models/sklearn/1.0/model</pre> | <pre>apiVersion: serving.kserve.io/v1alpha2 kind: LLMInferenceService metadata:   name: facebook-opt-125m-single spec:   model:     uri: hf://facebook/opt-125m     name: facebook/opt-125m</pre> |

# Generative AI with KServe

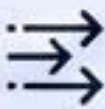


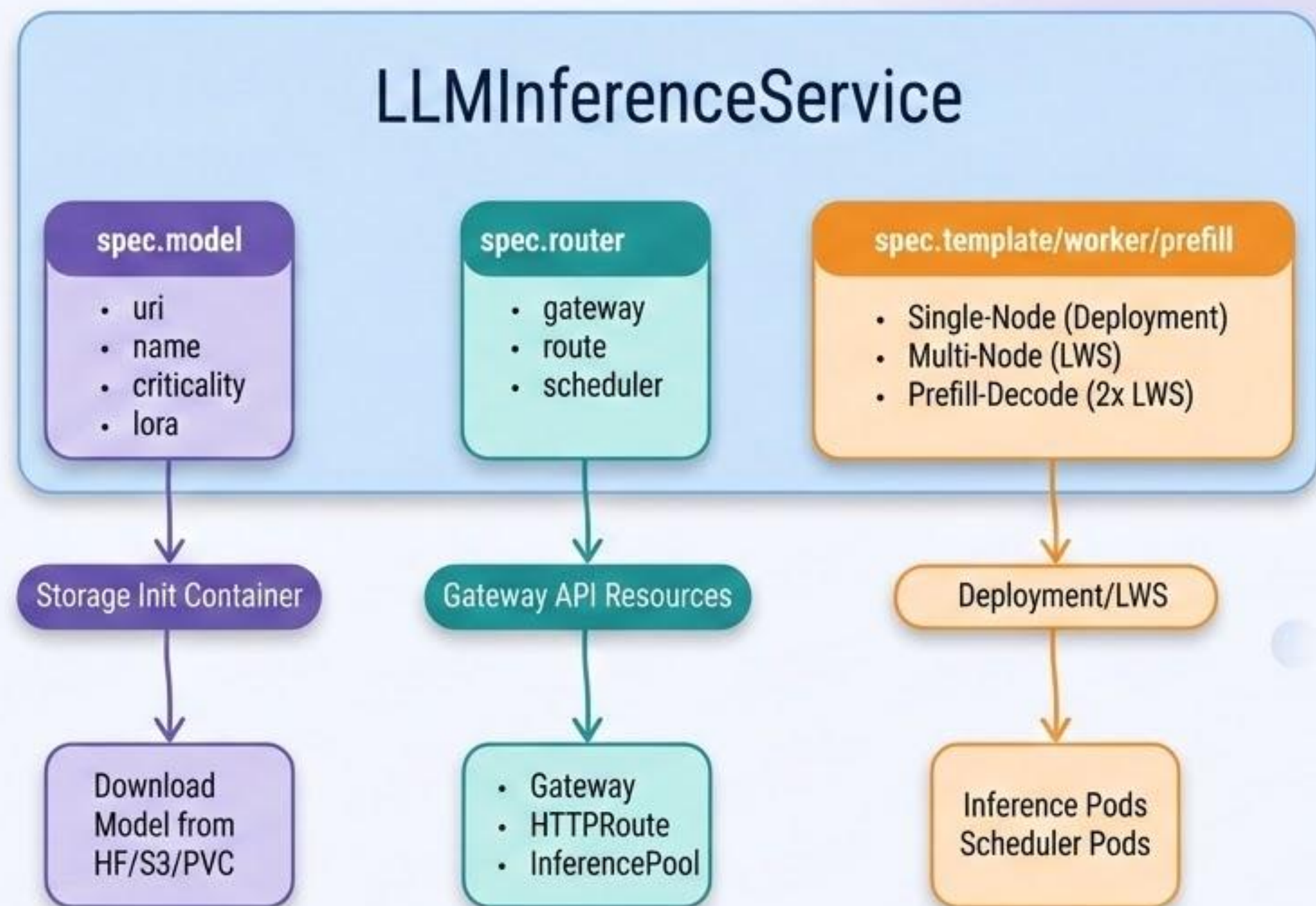
# LLMInferenceService Control Plane API

 **Model Spec:** Model URL, LORA

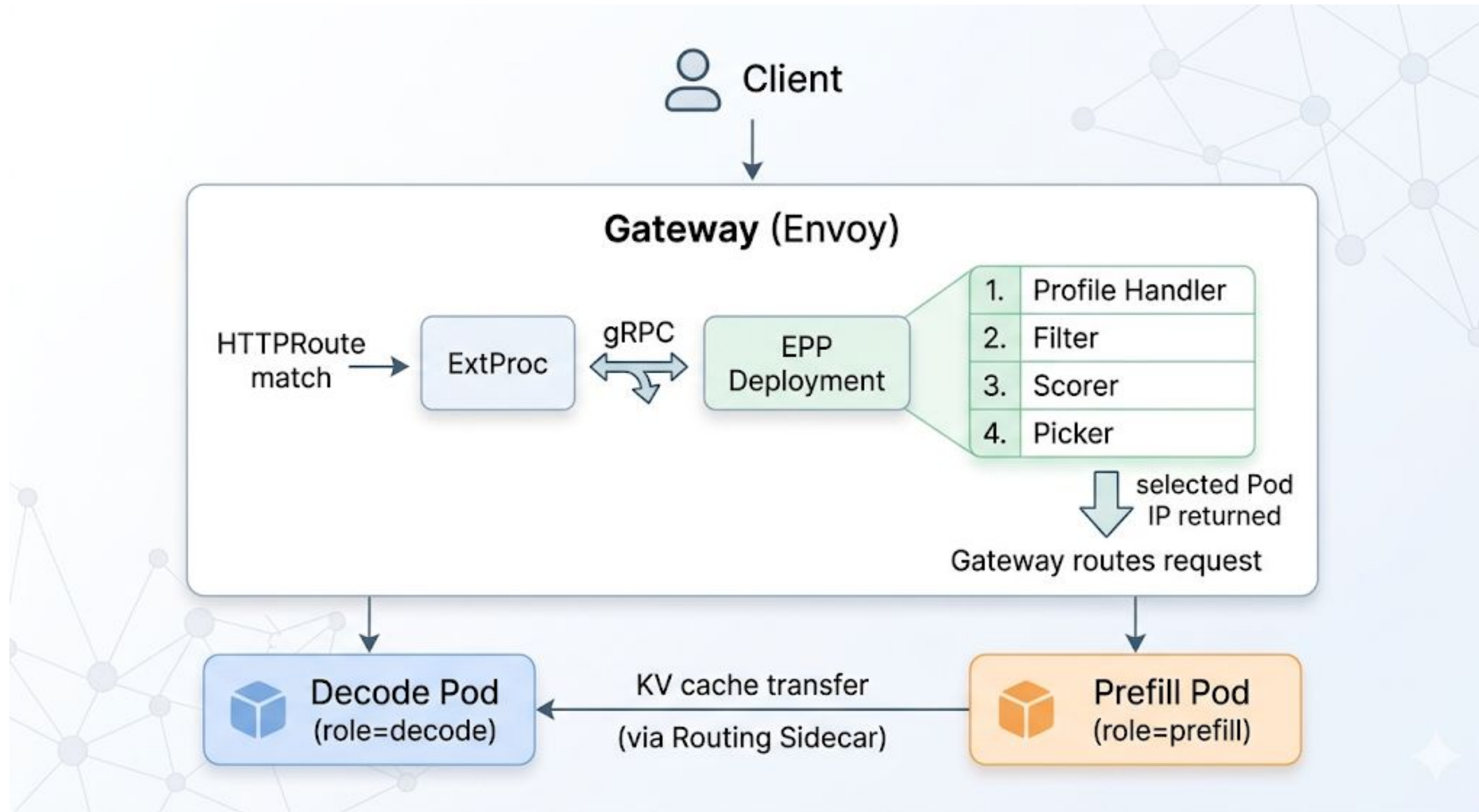
 **Router Spec:** Gateway API, Scheduler

 **Workload Spec:** Single/Multi Node, Prefill/Decode

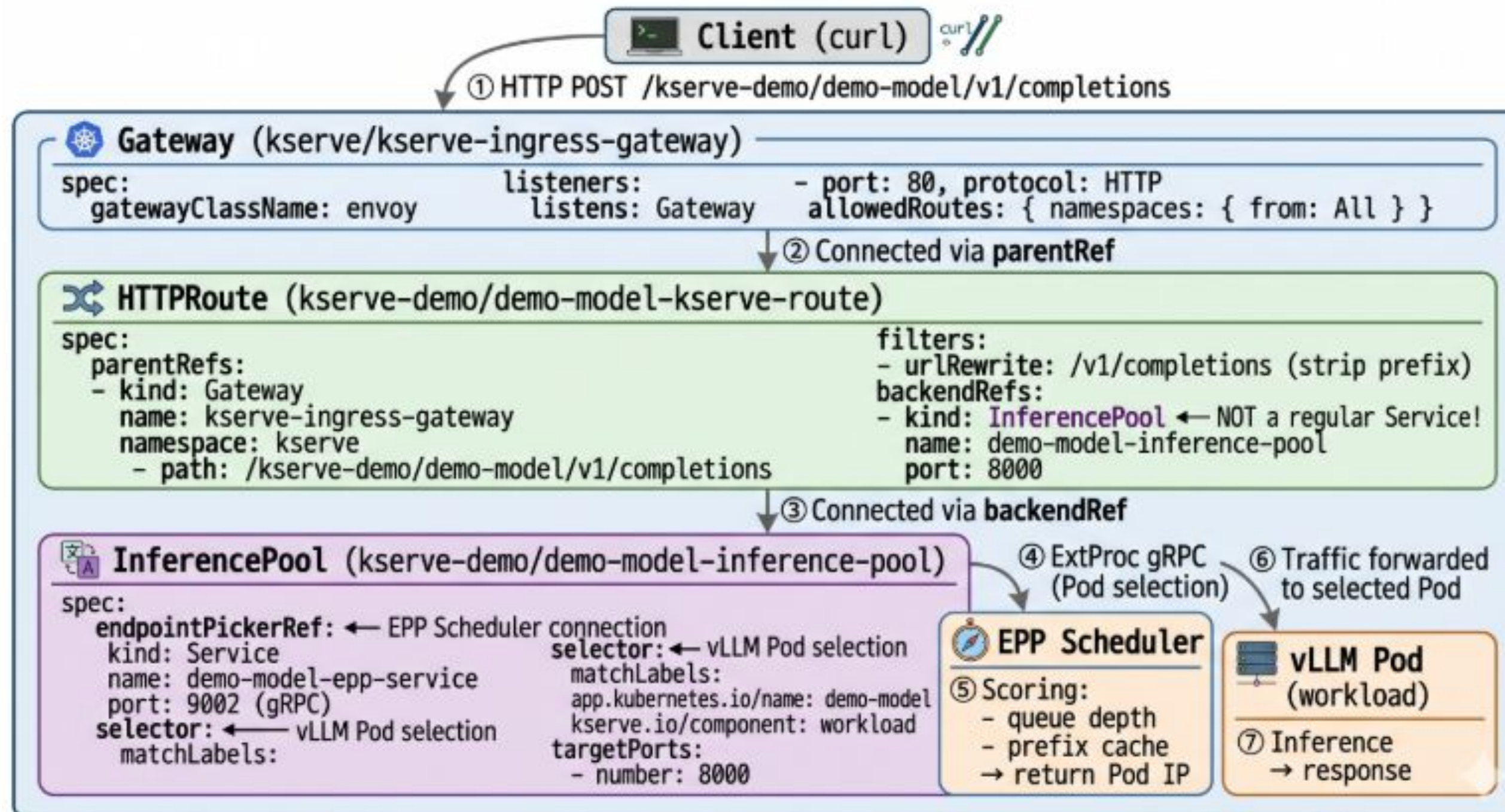
 **Parallelism Spec:** TP/DP/EP



# High Level Network Flow with KServe(P/D)



# Network Flow Through the Gateway API



03

# Workload Variant Autoscaler

LLM Specific Autoscaler

---

# WVA: LLM Workload-Aware Autoscaling

## Traditional

CPU, Memory, HTTP RPS

if cpu > 80 %: scale\_up()

if memory > 80%: scale\_up()

## LLM Specific

KV cache , GPU utilization, Queue, etc

if avg\_space\_kv < trigger\_threshold OR

avg\_spare\_queue < trigger\_threshold: scale\_up()

# WVA: Independent Scaling for Each component

## Prefill

```
prefill:  
  scaling:  
    minReplicas: 1  
    maxReplicas: 4  
  wva:  
    keda:  
      pollingInterval: 15  
      cooldownPeriod: 120
```

## Decode

```
scaling:  
  minReplicas: 2  
  maxReplicas: 8  
wva:  
  keda:  
    pollingInterval: 15  
    cooldownPeriod: 300
```

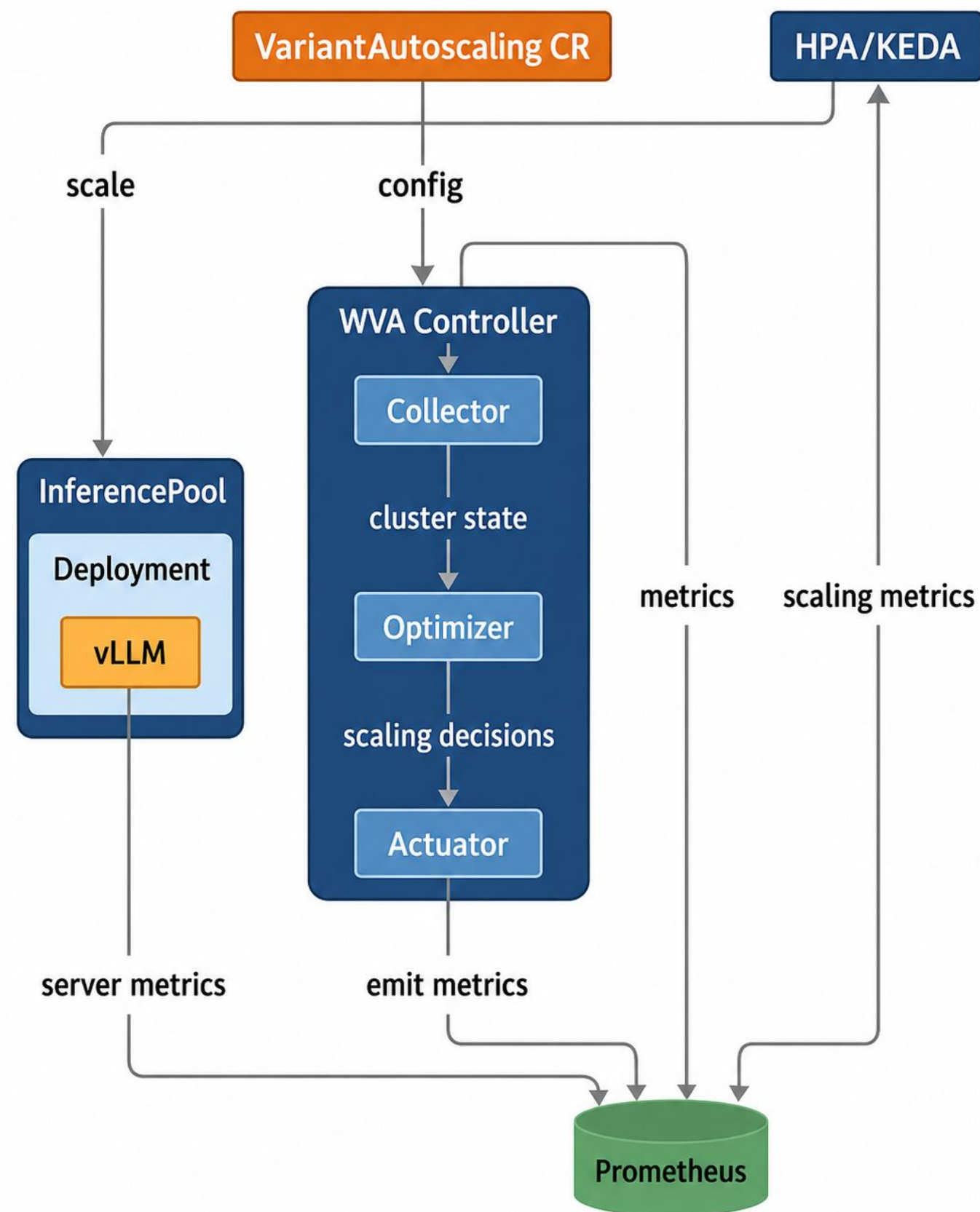
```
apiVersion: serving.kserve.io/v1alpha2
kind: LLMInferenceService
metadata:
  name: demo-model
spec:
  model:
    uri: "hf://facebook/opt-125m"
    name: facebook/opt-125m
  scaling:
    minReplicas: 1
    maxReplicas: 3
    wva:
      keda:
        pollingInterval: 15
        cooldownPeriod: 120
```

# Example

| Parameter                        | Value |                             |
|----------------------------------|-------|-----------------------------|
| Description                      |       |                             |
| scaling.minReplicas              | 1     | Maintain at least 1 Pod     |
| scaling.maxReplicas              | 3     | Scale out up to 3 Pods      |
| scaling.wva.keda.pollingInterval | 15    | Check metrics every 15s     |
| scaling.wva.keda.cooldownPeriod  | 120   | Wait 120s before scale down |

# Deploy WVA per Namespace

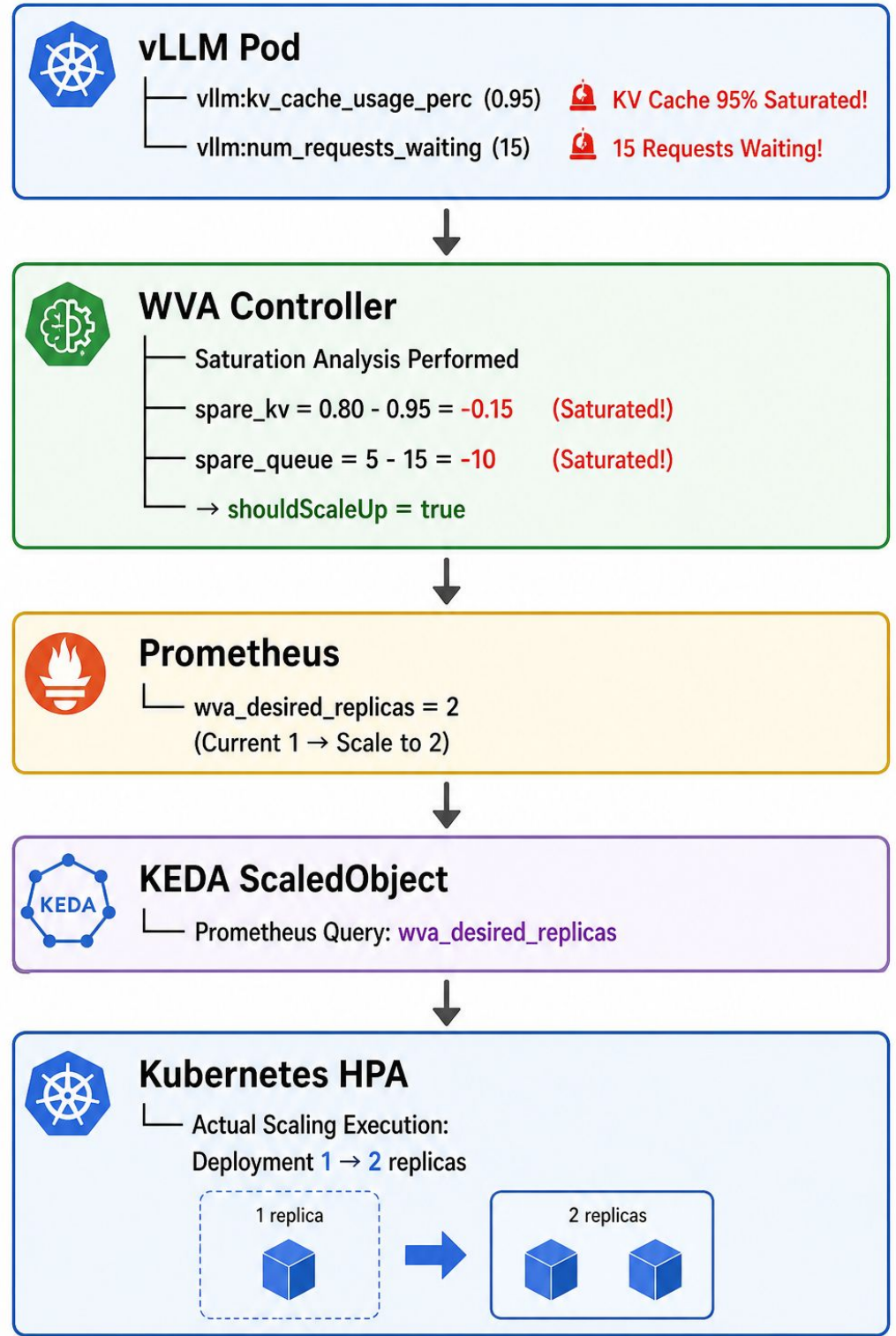
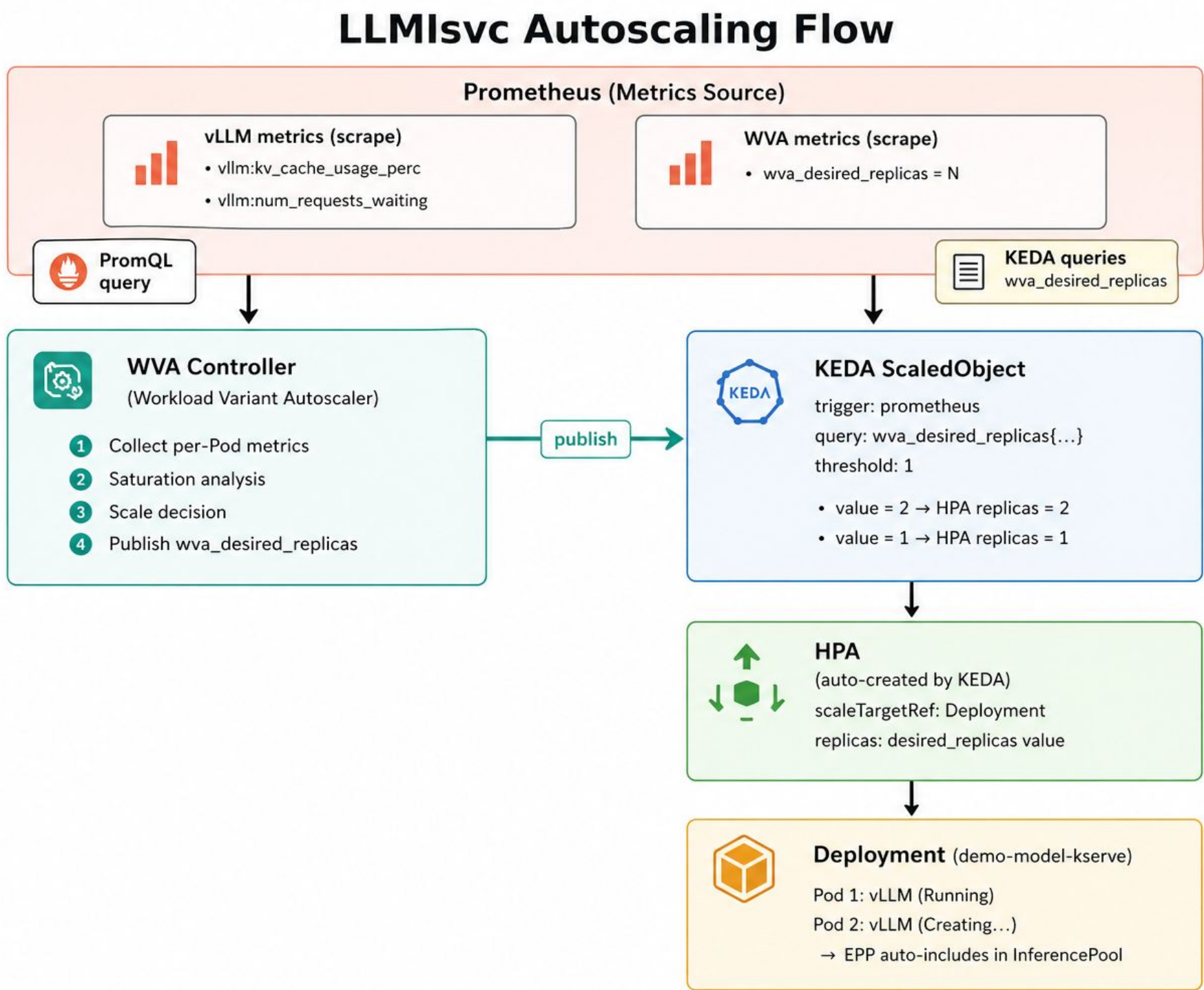
```
helm install wva "$WVA_DIR/charts/workload-variant-autoscaler" \
  --namespace $NS \
  --set controller.enabled=true \
  --set wva.replicaCount=1 \
  --set wva.image.tag=v0.6.0 \
  --set wva.namespaceScoped=true \
  --set wva.metrics.secure=false \
  --set wva.prometheus.baseURL="https://prom-tls-proxy.monitoring.svc:9443" \
  --set wva.prometheus.tls.insecureSkipVerify=true \
  --set wva.prometheus.monitoringNamespace=monitoring \
  --set wva.prometheus.serviceAccountName=prometheus-kube-prometheus-prometheus \
  --set va.enabled=false \
  --set hpa.enabled=false \
  --set vllmService.enabled=false \
  --set llmd.namespace=$NS
```



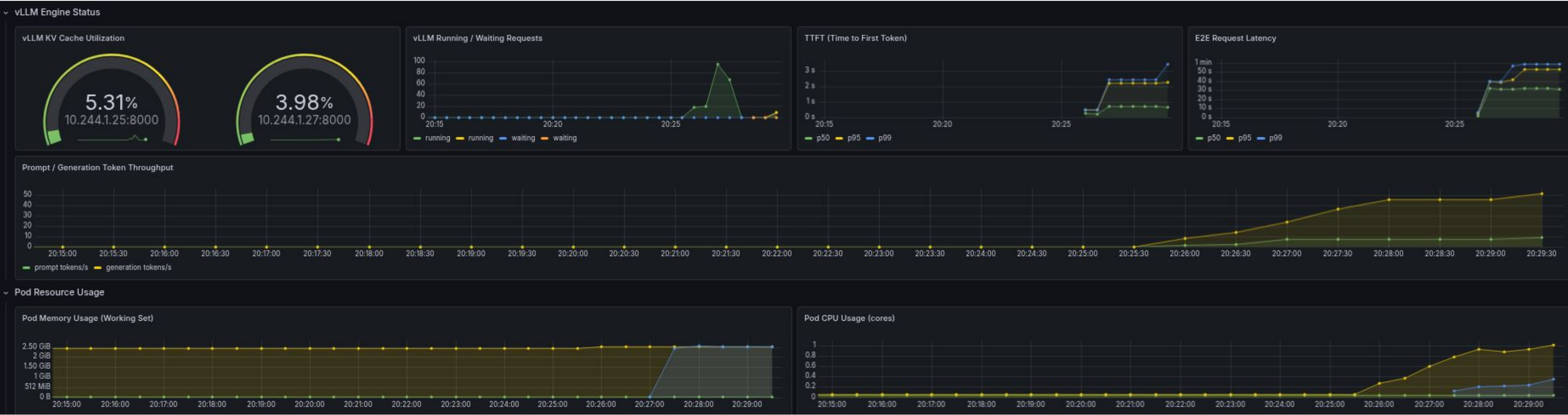
## How WVA work?

1. VariantAutoscaling CR is created and reconciled by the WVA controller
2. WVA monitors request rates and server performance via Prometheus metrics
3. The capacityCapacity model obtains KV cache utilization and queue depth of inference servers with slack capacity to determine replicas
4. Actuator emits optimization metrics to Prometheus and updates VA status
5. External autoscaler (HPA/KEDA) reads the metrics and scales the deployment (modelserver ONLY) accordingly

# Detailed WVA Workflow



# Sample Grafana Dashboard



# KServe Reference

- kserve website: <https://kserve.github.io/website/>
- kserver github: <https://github.com/kserve/kserve>
- KServe CNCF slack: <https://cloud-native.slack.com/archives/C06AH2C3K8B>

User Research: Model Serving  
Experience with KServe



# Thank You!

Jooho Lee

@Jooho · ljhiyh@gmail.com